

RAG 取り込みパイプライン 設計ノート

第一章 設計の前提

1.1 目的とスコープ

本書はテキスト埋め込み済みの日本語書籍を入力として、検索対象となるベクトルインデックスを構築する取り込みパイプラインの設計をまとめたものである。対象は抽出から格納までであり、回答生成そのものはここでは扱わない。各層は中間成果物をファイルとして残し、後段の再実行コストを最小化する。

層を分離する理由は明快である。抽出ロジックの変更は最も重い処理のやり直しを招くため、抽出結果を正本として保存しておけば、チャンク分割や埋め込みの調整はその正本を読み直すだけで済む。これにより試行錯誤の速度が大きく向上する。

1.2 実行基盤の方針

開発はローカルで完結させ、本番はクラウドへ移行する二段構えとする。両環境で中間成果物の形式を共通化することで、移行時の差分を埋め込みモデルと接続先の二点だけに閉じ込めることができる。スキーマや分割ロジックは共通である。

第二章 抽出の詳細

2.1 読み順の安定化

段組みや脚注を含む紙面では、素朴な抽出では読み順が乱れることがある。ブロック単位で取得し、座標に基づいて整列させることで、人間が読む順序に近いテキスト列を得る。これが後段のチャンク分割の品質を左右する。

見出しの検出には、紙面で最も多く使われるフォントサイズを本文とみなし、それより大きいものを見出しとする相対判定を用いる。加えて、章を表す定型の表現を併用することで、フォント情報が乏しい場合でも構造を復元しやすくなる。

2.2 正規化

段落の途中で入った改行は結合し、空行を段落の区切りとして扱う。ページ番号や繰り返し現れる書名などのヘッダ・フッタは、位置とパターンに基づいて取り除く。こうして得た整形済みテキストを正本として保存する。

2.3 コードブロックの扱い

技術書には実装例としてコードが掲載される。コードはモノスペースフォントで組まれるため、フォントフラグによって本文と区別できる。以下に例を示す。

```
def embed(texts):  
    return model.encode(texts)  
  
result = embed(['test'])
```

上記の関数はテキストのリストを受け取り、埋め込みベクトルを返す。コードブロックは分割せずに1チャンクとして扱う。